

RetailEdge Inc. — AWS Architecture

HardenedThree-Tier Design

This document is the updated AWS architecture for RetailEdge Inc.'s e-commerce platform. It incorporates the full set of required security, reliability, and operational changes on top of the original three-tier design: a fully private application tier reached with no NAT Gateway, WAF and CloudFront edge protections, a hardened Redis caching layer, least-privilege IAM, GitHub OIDC and Blue/Green CI/CD, remote Terraform state, and end-to-end logging and monitoring.

1. Architecture Overview

The solution remains organized into three layers, now hardened end-to-end:

- Web / Edge Tier — Route 53, CloudFront (with distinct static/API cache behaviors), and AWS WAF in front of the public entry point.
- Application Tier — Auto-scaled EC2 instances in private subnets, reached only through VPC Endpoints (no NAT Gateway), running the application as a Docker container with a Redis cache-aside layer.
- Data Tier — RDS MySQL (Multi-AZ), ElastiCache Redis (Multi-AZ, encrypted), and S3, all private and reachable only from the application tier.

2. High-Level Diagram

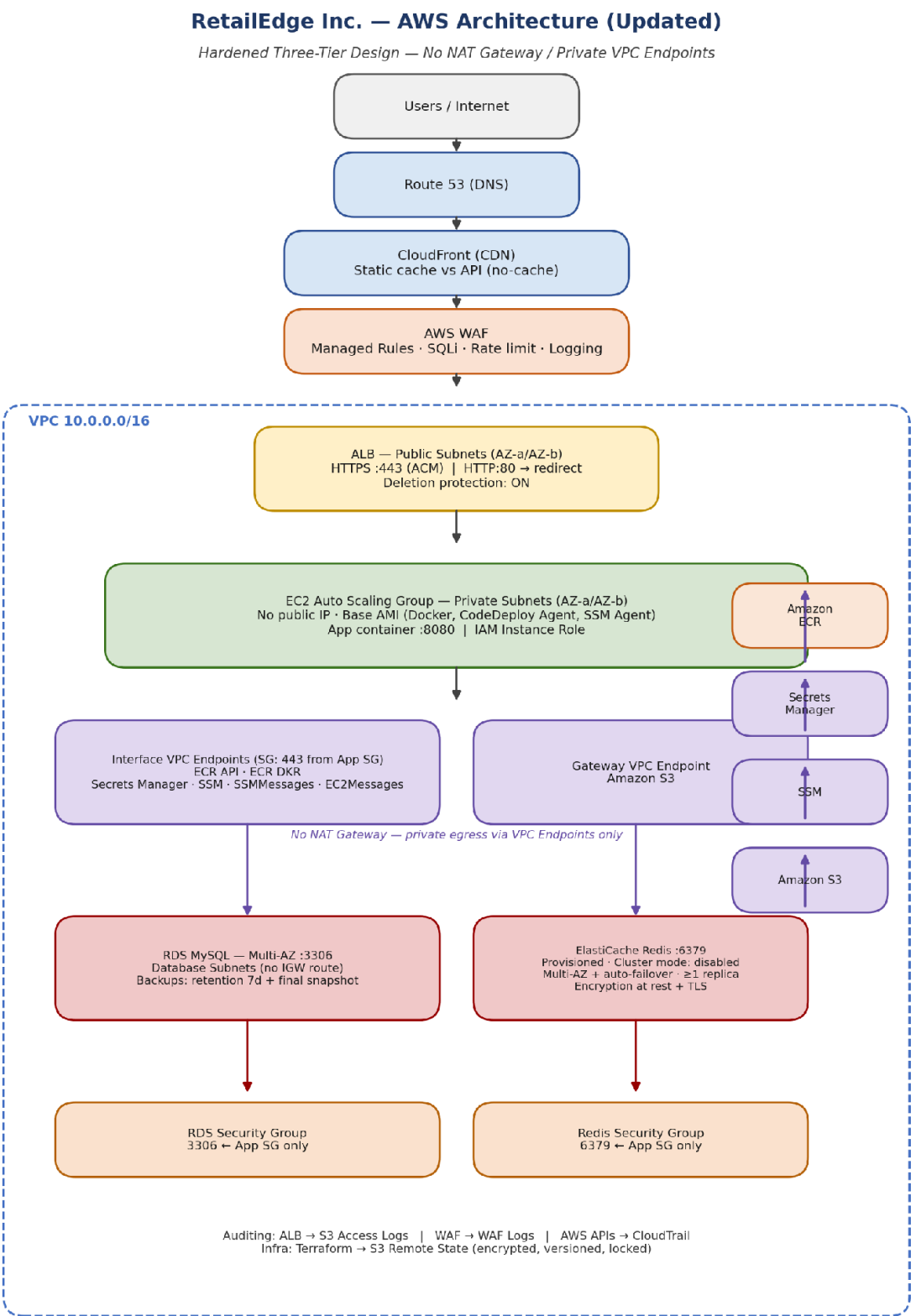


Figure 1 — High-level AWS architecture

3. Web / Edge Tier

- Amazon Route 53 provides DNS resolution for the application domain.
- CloudFront serves as the CDN layer in front of the ALB, with separate cache behaviors:
 - Static content (/static/*, /images/*, *.css, *.js) — cached with a long TTL.
 - Dynamic API (/api/*) — user-specific/dynamic responses are not cached at CloudFront; application-level caching is handled by Redis instead.
- ACM issues the certificate for the application domain. ALB :443 serves HTTPS using this certificate; ALB :80 only exists to redirect to :443. No normal application traffic is served over HTTP.
- ALB deletion protection is enabled in production (deletion_protection = true) to prevent accidental deletion of the main ALB.

4. Application Tier

- EC2 Auto Scaling Group instances sit in private subnets, have no public IPs, and are not directly reachable from the internet — only the ALB can reach them.
- Base AMI contains only infrastructure dependencies: Docker, CodeDeploy Agent, SSM Agent, and required OS configuration. It never contains DB/Redis passwords, AWS access keys, or application secrets.
- Application releases are decoupled from the AMI: CodeDeploy pulls the container image from ECR and starts it (Blue/Green strategy).
- Each instance uses a least-privilege IAM Instance Role (RetailEdgeAppInstanceRole) to reach Secrets Manager, ECR, and SSM — no long-lived AWS credentials are stored on the instance.
- Administration uses AWS SSM Session Manager (docker ps, docker logs, troubleshooting) — SSH port 22 is not exposed publicly.

5. Data Tier

5.1 Relational Database — Amazon RDS MySQL

- Multi-AZ for high availability; remains private with no public access.
- Automated backups are configured explicitly (backup_retention_period = 7); production deletions retain a final snapshot unless there is an explicit reason not to.
- The application uses a MySQL connection pool (bounded maximum connections) instead of opening a new connection per request, to avoid overwhelming RDS.

5.2 Caching — Amazon ElastiCache for Redis

- Deployment: Provisioned · Cluster Mode: Disabled · Multi-AZ: Enabled · Automatic Failover: Enabled · Replica: at least 1. Start with a small node and scale after measuring real usage.
- Encryption at rest and encryption in transit (TLS) are both enabled; if Redis auth is enabled, the auth token is stored in Secrets Manager (retailedge/redis) rather than in code.
- The Redis client is configured in RedisConfig.js; the endpoint and auth token are obtained at runtime, never hardcoded.

- The Redis client is created once at application startup and reused — a new connection is not opened per request.

5.3 Cache-Aside Pattern (TransactionService.js)

- Read path: check Redis first. On a cache hit, return the cached value directly with no database query. On a cache miss, read from RDS, return the data, and populate Redis with a configurable TTL (e.g. CACHE_TTL=60).
- Write path (POST / PUT-UPDATE / DELETE): after the RDS write succeeds, the affected Redis cache keys are invalidated (deleted) to prevent stale reads.
- Failure handling: Redis is a cache, not the source of truth. If Redis is unavailable, the application falls back to RDS directly and continues functioning, with degraded performance; the Redis client's reconnect/failover behavior is configured accordingly.

5.4 Object Storage — Amazon S3

- Default encryption (SSE-S3 or SSE-KMS) and Block Public Access are enabled on all application buckets.
- Versioning is enabled where appropriate, and bucket policies follow least privilege.

6. Network Architecture

A single VPC (10.0.0.0/16) spans two Availability Zones. The application tier no longer relies on a NAT Gateway for outbound access — private EC2 instances reach required AWS services entirely through VPC Endpoints.

Layer	Availability Zone A	Availability Zone B
Public	10.0.1.0/24	10.0.2.0/24
Application	10.0.11.0/24	10.0.12.0/24
Database	10.0.21.0/24	10.0.22.0/24

6.1 VPC Endpoints (replacing the NAT Gateway)

- Interface Endpoints: ECR API, ECR DKR, Secrets Manager, SSM, SSMMessages, EC2Messages (as required).
- Gateway Endpoint: S3.
- Interface endpoints use a dedicated Endpoint Security Group allowing TCP 443 from the Application Security Group only — endpoints are never exposed publicly.

7. Security Architecture

Security follows least privilege throughout: resources communicate only over the ports and with the components they explicitly require.

Security Group	Port	Allowed Source	Purpose
ALB SG	443 (HTTPS)	Internet (0.0.0.0/0)	Public entry point
ALB SG	80 (HTTP)	Internet (0.0.0.0/0)	Redirect to HTTPS only

Application SG	8080 (HTTP)	ALB SG only	App tier traffic (no 0.0.0.0/0 → 8080)
RDS SG	3306 (MySQL)	Application SG only	Database access, no public access
Redis SG	6379	Application SG only	Cache access, no internet access
Endpoint SG	443	Application SG only	Private access to ECR / Secrets Manager / SSM

7.1 IAM vs. Security Groups — Clarification

The EC2 instances do not need an IAM role "for RDS" or "for Redis." These are two distinct control planes:

- IAM Role — controls which AWS APIs the EC2 instance may call (EC2 → IAM Role → Secrets Manager: GetSecretValue).
- Security Groups — control which network connections are allowed (EC2 → TCP 3306 → RDS, and EC2 → TCP 6379 → Redis).

In short: IAM governs AWS permissions; Security Groups govern network permissions.

7.2 IAM Role for EC2 (RetailEdgeAppInstanceRole)

- Secrets Manager: secretsmanager:GetSecretValue, scoped to only the required DB/Redis secrets.
- ECR: ecr:GetAuthorizationToken, ecr:BatchGetImage, ecr:GetDownloadUrlForLayer, ecr:BatchCheckLayerAvailability.
- SSM: AmazonSSMManagedInstanceCore attached. No AWS access keys are placed on the EC2 instances.

7.3 Secrets Manager

- retailedge/db — { username, password, host, port: 3306 }.
- retailedge/redis — Redis authentication credentials, if Redis auth is enabled.
- Runtime retrieval: EC2 → IAM Role → Secrets Manager → DB/Redis credentials → application memory only. Secrets are never stored in GitHub, the Docker image, the Dockerfile, the AMI, source code, or a static .env baked into the image, and are never printed to logs.

8. Container and Deployment Architecture

The application artifact remains separate from the EC2 server image, so releases never require a new AMI.

Deployments now use a Blue/Green strategy with automatic rollback:

- Blue = current production, Green = new version.
- Deploy to Green → health validation against /health → on success, traffic shifts and Green becomes production.
- On failure, traffic is not shifted and the deployment rolls back, leaving Blue in production.
- ECR image scanning is enabled; images avoid embedded secrets, use secure/updated dependencies, use a minimal base image where appropriate, and run as non-root where practical.

9. CI/CD Architecture (Updated)

GitHub Actions authenticates to AWS via OIDC rather than long-lived access keys. The IAM trust policy is restricted to the exact repository and branch/ref, e.g. `repo:your-org/retailedge-app:ref:refs/heads/main` — not every repository in the organization can assume the production role.

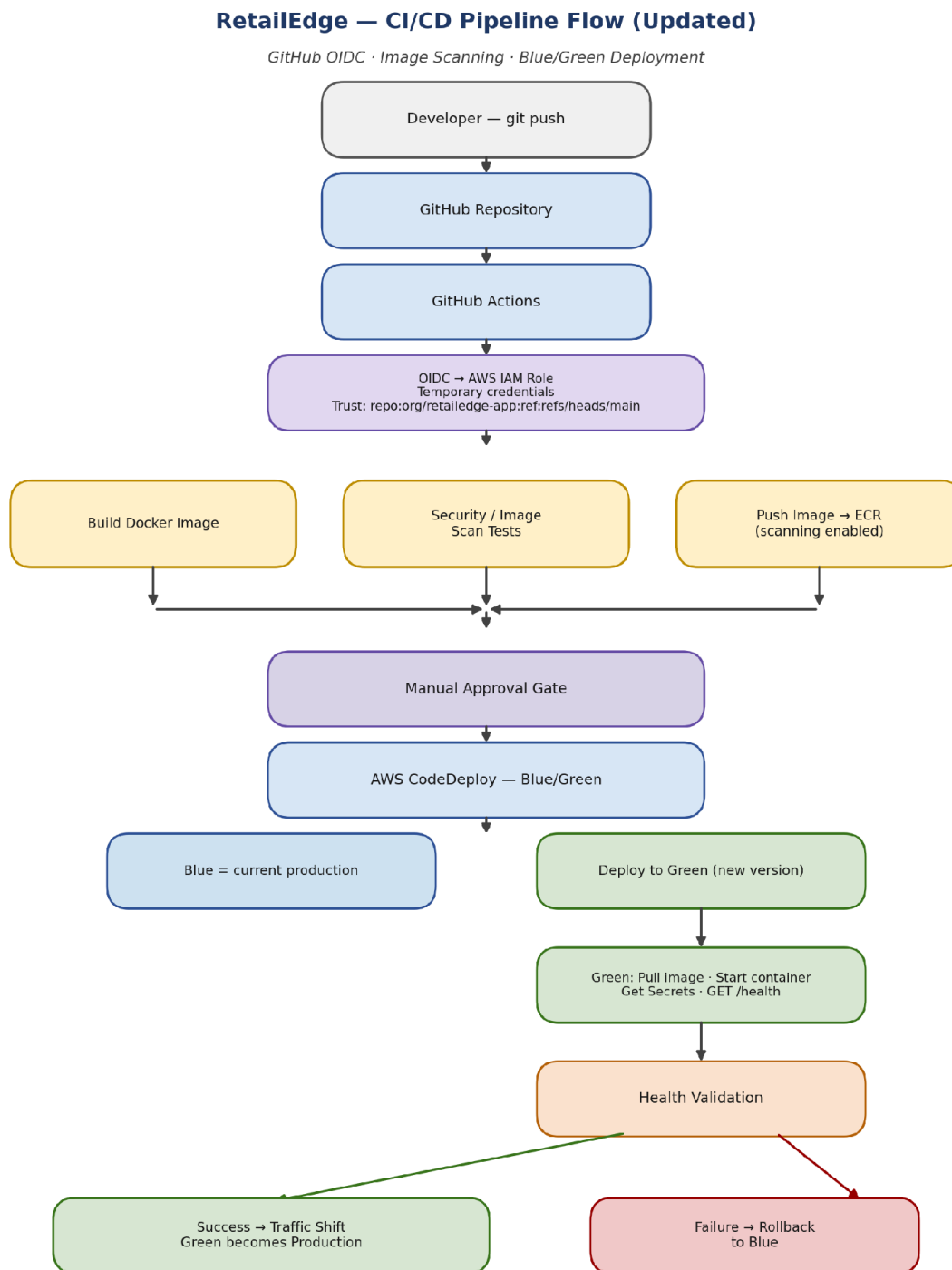


Figure 2 — CI/CD pipeline flow

Pipeline: Developer push → GitHub Actions → OIDC → temporary AWS credentials → build Docker image → security/image tests → push to ECR (scanning enabled) → manual approval → CodeDeploy Blue/Green → pull image on Green → get secrets → /health validation → traffic shift → production (or rollback to Blue on failure).

10. Infrastructure as Code

Terraform continues to own the infrastructure lifecycle. Shared production state is no longer local:

- Remote backend: Terraform → S3 backend, with the state bucket encrypted, blocking public access, versioned, and restricted to authorized IAM principals.
- State locking uses whichever locking mechanism the Terraform version in use supports (a dedicated DynamoDB table if using the traditional locking approach).
- Requirement in one line: remote, encrypted, protected, and locked Terraform state.

11. Monitoring, Logging, and Auditing (Updated)

Component	Key Metrics / Logs	Example Alarm / Use
ALB	Request count, target response time, 4xx, 5xx, healthy host count	P95 latency > 800ms for 5 min → SNS
EC2 / ASG	CPU, memory, network, instance & healthy-host status	Instance health alarms → SNS
RDS	CPU, connections, storage, read/write latency	CPU / connection alarms → SNS
Redis	CPU, memory, connections, hits/misses, evictions, hit ratio	Cache hit ratio alarms → SNS
ALB Access Logs	Path, status code, timing, client info → S3	Traffic investigation / troubleshooting
WAF Logging	Blocked/allowed requests, triggered rules, rate limiting	Investigate suspicious traffic, false positives
CloudTrail	Who / what action / when / which resource across AWS APIs	IAM, Secrets Manager, RDS, S3, SG, Terraform changes

12. Final Checklist

Network

- ✓ Private EC2 subnets, no public IPs
- ✓ VPC Endpoints instead of NAT (ECR API/DKR, Secrets Manager, SSM endpoints, S3 gateway)
- ✓ Dedicated Endpoint Security Group

Edge / Security

- ✓ Route 53, CloudFront (static/API cache behaviors), WAF (managed rules, rate limiting, logging)

- ✓ ACM certificate, HTTPS 443, HTTP→HTTPS redirect, ALB deletion protection

Security Groups

- ✓ ALB → App :8080, App → RDS :3306, App → Redis :6379 — no public access to App/RDS/Redis

IAM & Secrets

- ✓ Least-privilege EC2 instance role; no AWS access keys on EC2; GitHub OIDC with restricted trust policy
- ✓ DB and Redis credentials in Secrets Manager, retrieved at runtime; none in code, AMI, image, or Git

RDS & Redis

- ✓ Multi-AZ RDS, automated backups + retention + final snapshot policy, connection pooling, parameterized queries
- ✓ Redis: Multi-AZ, replica, automatic failover, encryption at rest + TLS, cache-aside, TTL, invalidation, failure fallback, connection reuse

Deployment & CI/CD

- ✓ Docker, CodeDeploy Agent, SSM Agent on AMI; Blue/Green deployment; /health validation; automatic rollback
- ✓ GitHub Actions + OIDC, image scanning, manual approval, CodeDeploy Blue/Green

S3, Terraform, and Observability

- ✓ S3 encryption, Block Public Access, versioning, least-privilege IAM
- ✓ Remote encrypted/versioned/locked Terraform state in S3
- ✓ ALB access logs, WAF logs, CloudTrail, CloudWatch alarms including cache hit/miss ratio